

# Lagrangian 2D Fluid Simulation Using Smoothed Particle Hydrodynamics (SPH)

*A Project Component for*  
**Parallel and Distributed Computing (UCS645)**

*By*

| Sr | Name               | Roll No   |
|----|--------------------|-----------|
| 1  | Bhawesh War        | 102313065 |
| 2  | Ketanpreet Singh   | 102303386 |
| 3  | Ayush Pratap Singh | 102303974 |
| 4  | Ishaan Singh       | 102303382 |

*Under the guidance of*  
**Dr. Saif Nalband**  
(Assistant Professor, DCSE)



**THAPAR INSTITUTE**  
OF ENGINEERING & TECHNOLOGY  
(Deemed to be University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING  
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY  
(DEEMED TO BE UNIVERSITY)  
PATIALA - 147004

**MAY, 2025**

# Contents

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>3</b>  |
| 1.1      | Background . . . . .                                       | 3         |
| 1.2      | Introduction to Problem Statement . . . . .                | 4         |
| <b>2</b> | <b>Literature Review</b>                                   | <b>5</b>  |
| <b>3</b> | <b>Research Gaps</b>                                       | <b>6</b>  |
| <b>4</b> | <b>Problem Formulation</b>                                 | <b>6</b>  |
| <b>5</b> | <b>Objectives</b>                                          | <b>7</b>  |
| <b>6</b> | <b>Methodology</b>                                         | <b>8</b>  |
| 6.1      | The Smoothing Kernel and Density Smoothing . . . . .       | 8         |
| 6.2      | Numerical Time Integration . . . . .                       | 9         |
| 6.3      | Algorithmic Optimization: The Linked-Cell Method . . . . . | 9         |
| <b>7</b> | <b>Datasets</b>                                            | <b>10</b> |
| <b>8</b> | <b>Results and Discussion</b>                              | <b>10</b> |
| 8.1      | Physical Validation and Wave Kinematics . . . . .          | 11        |
| 8.2      | Spatial Convergence Study . . . . .                        | 11        |
| 8.3      | Computational Efficiency Summary . . . . .                 | 12        |
| <b>9</b> | <b>High Performance Computational Analysis</b>             | <b>12</b> |
| 9.1      | OpenMP: Shared-Memory Performance Analysis . . . . .       | 12        |
| 9.2      | MPI: Distributed-Memory Scaling . . . . .                  | 13        |
| 9.3      | Hardware Telemetry (perf) Analysis . . . . .               | 13        |
| 9.4      | CUDA: Hardware-Accelerated Analysis . . . . .              | 14        |

# 1

## Introduction

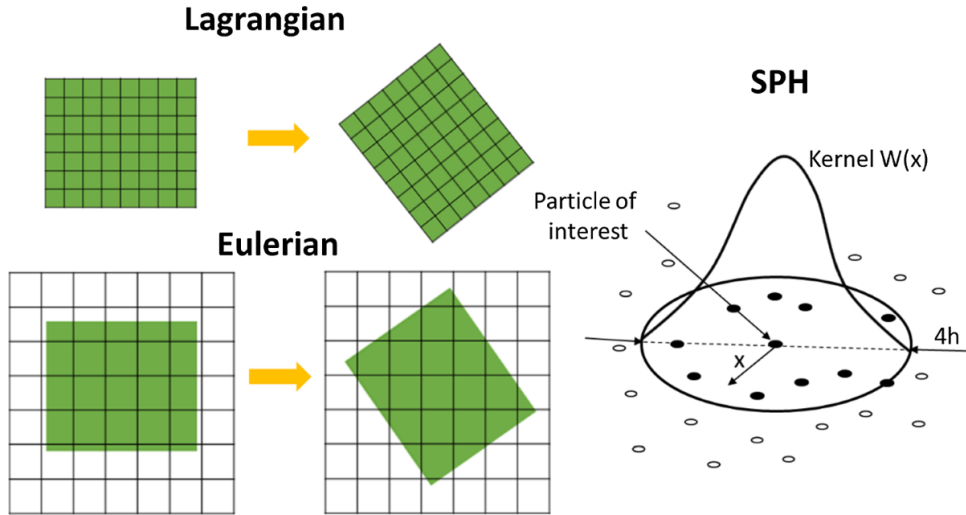
### 1.1

#### Background

Smoothed Particle Hydrodynamics (SPH) is a powerful, meshless computational method primarily utilized for solving the Navier-Stokes equations. Unlike traditional Eulerian methods that rely on a fixed spatial grid to calculate fluid dynamics, SPH is a Lagrangian approach, meaning the computational reference frame actively follows the fluid motion. This fundamental characteristic makes SPH exceptionally well-suited for simulating highly dynamic problems involving fluid flow with complex interfaces, large deformations, and free surfaces, such as breaking waves or splashing.

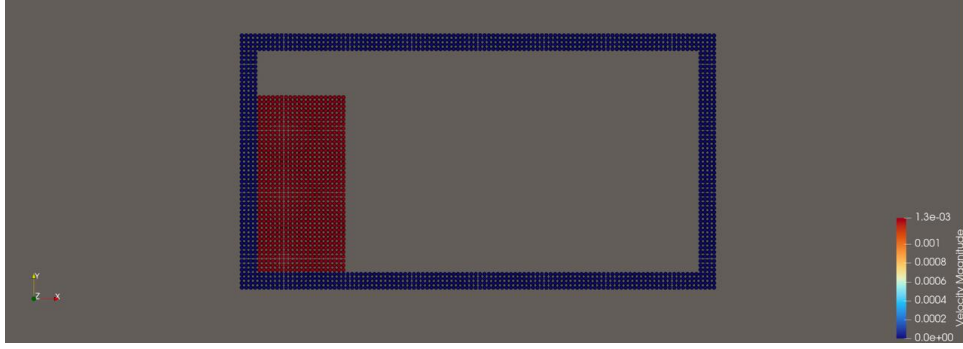
At its core, the SPH method discretizes the fluid continuum into a finite set of individual, moving points, commonly referred to as "particles". Each of these particles is associated with a specific mass of the fluid,  $m_i$ , and carries physical properties such as velocity, density, and pressure. Because there is no mesh to connect these points, SPH calculates the continuous macroscopic fields by interpolating the discrete values of surrounding particles.

This interpolation is achieved through the use of a "smoothing kernel," mathematically denoted as  $W(x - x_j, h)$ . The smoothing kernel is a weighting function that determines how strongly a particle is influenced by its neighbors, scaled by a characteristic smoothing length,  $h$ . A critical property of this kernel is its "compact support," which ensures that the weighting function drops to zero at a finite distance. Consequently, only particles within a localized neighborhood (typically a radius of  $2h$ ) contribute to the calculations, preventing the need to compute interactions across the entire fluid domain. In 2D simulations, Cubic Spline Kernel is generally employed.



**Figure 1:** A conceptual comparison of traditional mesh-based fluid dynamics versus the meshless, particle-based SPH method.

## 1.2 Introduction to Problem Statement

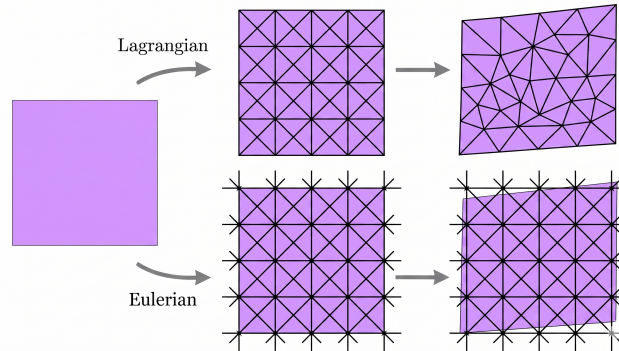


**Figure 2:** *The initial problem geometry, featuring the fixed boundary particles (walls) and the suspended fluid particles prior to the wave collapse.*

In this project, our primary focus is to build a functional SPH simulator from scratch to model a dynamic wave generation problem. We are essentially tracking a scenario where a column of water collapses inside a constrained 2D tank, meaning we have to carefully calculate the changing velocity, density, and pressure of thousands of individual fluid particles over time.

Mathematically, this boils down to approximating the Navier-Stokes and continuity equations. However, setting up these equations presents an immediate structural problem: we are left with four unknowns for every single particle (velocity in the  $x$  and  $y$  directions, pressure, and density), but we only have three core equations to solve them. To bridge this gap and close the system, we treat the water as a slightly compressible fluid. By introducing the stiff Tait equation of state, we can directly link the fluid's pressure to its local density, allowing the math to resolve.

Finally, we have to tackle the physical boundaries. In a meshless Lagrangian framework, creating a solid wall is surprisingly tricky. Without proper constraints, high-pressure fluid particles will simply leak out of the domain or clump unnaturally at the edges. We formulate our tank boundaries by building a perimeter made of fixed boundary particles. While these wall particles are locked in place with zero velocity.



**Figure 3:** *Comparison of Lagrangian and Eulerian Meshes.*

## 2

Literature ReviewTable 1: *Summary of Literature Review*

| References                   | Concepts Highlighted                                                  | Technique(s) used                                                                 | Significance/ Proposal                                                                                                                     | Limitations                                                                                                                           |
|------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Monaghan (1994) [1]          | Free surface flows; Wave simulation; SPH fundamentals                 | Lagrangian meshless formulation; Artificial viscosity; Tait equation of state.    | Established the baseline numerical method for using SPH to model incompressible fluids with free surfaces (like water waves).              | First-order accurate integrations can lead to numerical dissipation; requires very small timesteps to maintain stability.             |
| Becker & Teschner (2007) [2] | Weakly Compressible SPH (WCSPH); Stiff equations                      | Tait equation with artificial speed of sound; Predictor-Corrector integration.    | Demonstrated that treating water as weakly compressible allows for explicit pressure calculations without solving heavy Poisson equations. | The artificial speed of sound severely restricts the maximum allowable timestep (CFL condition), causing high computational cost.     |
| Adami et al. (2012) [3]      | Boundary conditions; Wall interactions; Dummy particles               | Fixed boundary particles with dynamically calculated pressure and density fields. | Solved the issue of particle leakage and unphysical clumping by allowing boundary particles to interact physically with the fluid.         | Complex domain geometries require intricate boundary particle placement; sharp corners can still induce localized pressure anomalies. |
| Dominguez et al. (2011) [4]  | HPC Optimization; Neighbor Search algorithms; Memory caching          | Linked-Cell spatial hashing; GPU execution profiling; OpenMP threading.           | Proved that reorganizing particle data via spatial hashing reduces search complexity to $O(N)$ and maximizes cache hits for CPUs and GPUs. | Memory transfer bottlenecks between CPU and GPU over the PCI-e bus can limit overall scaling if not carefully managed.                |
| Crespo et al. (2015) [5]     | Parallel CFD solvers; Distributed Memory scaling; Multi-GPU execution | MPI domain decomposition; CUDA hardware acceleration.                             | Created a highly scalable framework demonstrating that hybrid MPI+CUDA approaches are essential for multi-million particle domains.        | MPI communication overhead increases significantly across multiple nodes, limiting perfect linear scalability for dense fluids.       |

### 3 Research Gaps

While the fundamental mathematical framework of Smoothed Particle Hydrodynamics (SPH) is well-established for free-surface flows, transitioning from theoretical equations to a high-performance, scalable simulator reveals several distinct computational and physical roadblocks. Based on the current literature, the primary research gaps addressed in this project include:

- **High-Frequency Pressure Oscillations and Stability:** Modeling water as weakly compressible requires the use of the highly stiff Tait equation. While higher-order integration schemes (like Predictor-Corrector) mitigate basic temporal instability, the fluid remains highly susceptible to localized pressure spikes and unphysical particle clustering, requiring highly tuned density smoothing algorithms to maintain volume conservation.
- **Boundary-Induced Artifacts:** Traditional methods for enforcing solid boundaries (such as dummy or ghost particles) successfully prevent fluid leakage, but they introduce significant memory overhead. Furthermore, these boundary configurations often struggle at complex domain geometries (like sharp 90-degree corners), where they can induce false pressure anomalies and unnatural fluid friction.
- **CPU-GPU Memory Transfer Bottlenecks:** Algorithmic optimizations like the Linked-Cell spatial hashing method effectively reduce neighbor search complexity from  $O(N^2)$  to  $O(N)$ . However, when deploying these algorithms on hardware accelerators (GPUs), a severe architectural gap emerges. The constant transfer of reorganized particle data across the PCI-e bus acts as a massive physical bottleneck, often neutralizing the speedup gained from CUDA cores if not managed perfectly.
- **Distributed Memory Communication Overhead:** Scaling a fluid simulation to multi-million particle domains requires distributed memory architectures (MPI). The critical gap in linear scalability lies in the halo regions—the edges of the divided domain chunks. As the simulation scales across multiple nodes, the network communication time required to synchronize these boundaries rapidly outpaces the actual mathematical computation time.

### 4 Problem Formulation

To accurately simulate our fluid, we must translate the governing laws of fluid dynamics—specifically, the Navier-Stokes and continuity equations—into a language our particle-based framework can process. In a Lagrangian system, we are tracking the derivatives of fluid motion within the reference frame of the moving particles themselves.

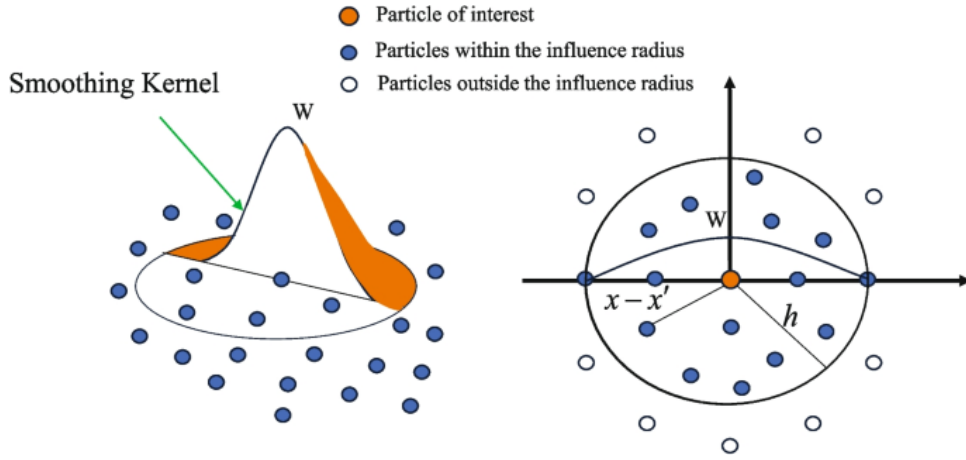
The primary challenge in formulating this problem is ensuring that mass and momentum are explicitly conserved. To achieve this, we approximate the partial derivatives of the fluid using symmetric and anti-symmetric kernel gradients. This results in the following discrete approximations for the momentum (acceleration) and continuity (density change) equations:

$$a_i = - \sum_{j=1}^N m_j \left( \frac{P_i}{\rho_i^2} + \frac{P_j}{\rho_j^2} \right) \frac{dW}{dr} e_{ij} + \mu \sum_{j=1}^N m_j \left( \frac{1}{\rho_i^2} + \frac{1}{\rho_j^2} \right) \frac{dW}{dr} \frac{v_{ij} \cdot e_{ij}}{|r_{ij}|} + g \quad (1)$$

$$D_i = \sum_{j=1}^N m_j \frac{dW}{dr} v_{ij} \cdot e_{ij} \quad (2)$$

However, setting up these equations introduces an immediate structural issue: for every fluid particle, we have four unknowns (velocity in the  $x$  and  $y$  directions, pressure, and density), but only three equations to solve them. To bridge this gap and mathematically close the system, we treat the water as a slightly compressible fluid. By introducing the stiff Tait equation of state, we establish a direct, computable link between a particle's local density and its pressure:

$$P = B \left( \left( \frac{\rho}{\rho_0} \right)^\gamma - 1 \right) \quad (3)$$



**Figure 4:** A visual representation of particle interactions within the smoothing length  $h$ , demonstrating how pressure and viscous forces are calculated between a central particle and its neighbors.

Finally, we must formulate a stable interaction model for the physical boundaries of the tank. In a meshless framework, creating a solid wall is surprisingly tricky; without proper constraints, high-pressure fluid particles will leak out of the computational domain or clump unnaturally. We solve this by constructing a perimeter made of fixed boundary particles, exactly  $2h$  wide. While these wall particles are locked in place with zero velocity, their density and pressure dynamically evolve just like the fluid particles. This creates a natural, physical repulsive barrier that safely contains the wave within the simulation space without destabilizing the math.

## 5 Objectives

The overarching goal of this project is to develop, validate, and aggressively optimize a C++ based Smoothed Particle Hydrodynamics (SPH) fluid simulator. To achieve this, the project is divided into the following concrete objectives:

- **Core Solver Development:** To program a functional 2D SPH simulator capable of resolving free-surface fluid dynamics and wave generation over a 30-second temporal domain.

- **Algorithmic Optimization:** To reduce the computational complexity of particle interactions from a naive  $O(N^2)$  to  $O(N)$  by implementing a highly efficient linked-cell spatial hashing algorithm for neighbor searching.
- **Numerical Integration Upgrade:** To improve the mathematical stability and accuracy of the simulation by transitioning from a first-order Forward Euler integration scheme to a second-order Predictor-Corrector time-stepping scheme.
- **Data Pipeline and Visualization:** To engineer a robust data-export pipeline that outputs particle states at user-defined intervals, paired with a custom post-processing workflow to render the fluid as an animation.
- **Physical Validation:** To validate the physical accuracy of the solver by tracking wave crest kinematics and sloshing intervals, comparing the simulated results against the theoretical shallow water wave speed ( $\sqrt{gh_0}$ ).
- **High-Performance Computing (HPC) Extension:** To extend the sequential solver into the parallel computing domain, systematically evaluating its scalability and hardware utilization using Shared-Memory (OpenMP), Distributed-Memory (MPI), and Hardware-Accelerated (CUDA) paradigms.

## 6 Methodology

Building a robust SPH simulator requires translating continuous physics into discrete, algorithmic steps. Our methodology centers on three major computational pillars: calculating the physical forces via a smoothing kernel, safely stepping the simulation forward in time, and optimizing the neighbor-search process to ensure the software runs efficiently.

### 6.1 The Smoothing Kernel and Density Smoothing

To calculate how particles influence one another, we utilize a 2D cubic spline smoothing kernel. Think of this as a "sphere of influence" around each particle; the closer a neighboring particle is, the stronger its mathematical weight. The cubic spline is chosen because it is symmetric and smoothly drops to zero at the boundary of our interaction radius ( $2h$ ), ensuring stable gradient calculations:

$$W(r, h) = \frac{10}{7\pi h^2} \begin{cases} 1 - \frac{3}{2}q^2 + \frac{3}{4}q^3 & \text{if } 0 \leq q \leq 1 \\ \frac{1}{4}(2 - q)^3 & \text{if } 1 \leq q \leq 2 \\ 0 & \text{if } q > 2 \end{cases} \quad (4)$$

Where  $q = \frac{|r|}{h}$ .

Additionally, because local variations in particle spacing can cause the density field to become artificially "rough" or noisy over time, we apply a mathematical smoothing pass to the density every 10 to 20 timesteps. This acts as a filter to prevent unphysical particle clumping:

$$\rho_i = \frac{\sum_{j=1}^n W(r_{ij}, h)}{\sum_{j=1}^n \frac{W(r_{ij}, h)}{\rho_j}} \quad (5)$$



## 6.2 Numerical Time Integration

If we simply push particles forward based on their current acceleration (a standard first-order Forward Euler approach), the simulation quickly becomes unstable unless we use impossibly microscopic time steps. To guarantee mathematical stability and achieve second-order accuracy, we implemented a Predictor-Corrector integration scheme.

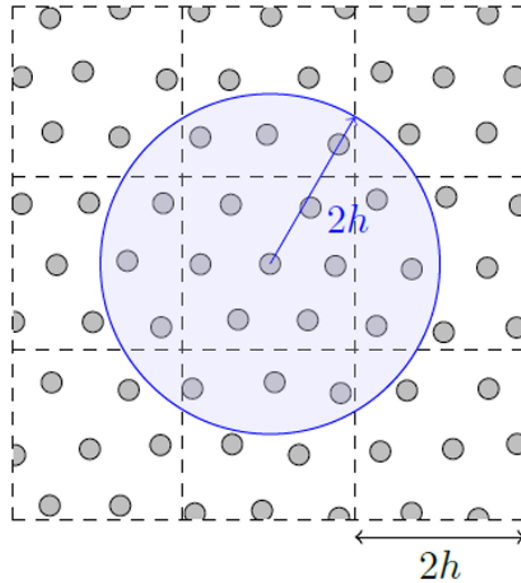
In this approach, the algorithm "looks ahead" by calculating a half-step prediction of where the particles will be, re-evaluates the forces at that future position, and then uses that data to make a highly accurate full-step correction. The position updates are governed by:

$$\text{Half-Step: } x_i^{t+1/2} = x_i^t + 0.5\Delta t v_i^t \quad \text{Full-Step: } x_i^{t+1} = 2\overline{x}_i^{t+1/2} - x_i^t \quad (6)$$

## 6.3 Algorithmic Optimization: The Linked-Cell Method

Naively checking the distance between every single particle in the tank to see if they are close enough to interact results in an algorithmic complexity of  $O(N^2)$ . For a simulation with thousands of particles, this would grind the CPU to a halt.

To optimize this, we utilize a Linked-Cell spatial hashing method. At the beginning of every timestep, the entire simulation domain is divided into a background grid of square cells, each exactly  $2h$  wide. Particles are binned into their respective cells. When it is time to calculate forces, a particle no longer searches the whole tank; it only checks the particles within its own cell and the immediately adjacent neighboring cells. Because the number of particles in a local cell is generally constant, the time it takes to compute forces for a single particle becomes  $O(1)$ , dropping the overall simulation complexity to a highly efficient  $O(N)$ .



**Figure 5:** The Linked-Cell neighbor search algorithm. The domain is divided into cells of width  $2h$ , restricting distance calculations only to a particle's immediate adjacent cells, drastically improving computational performance.

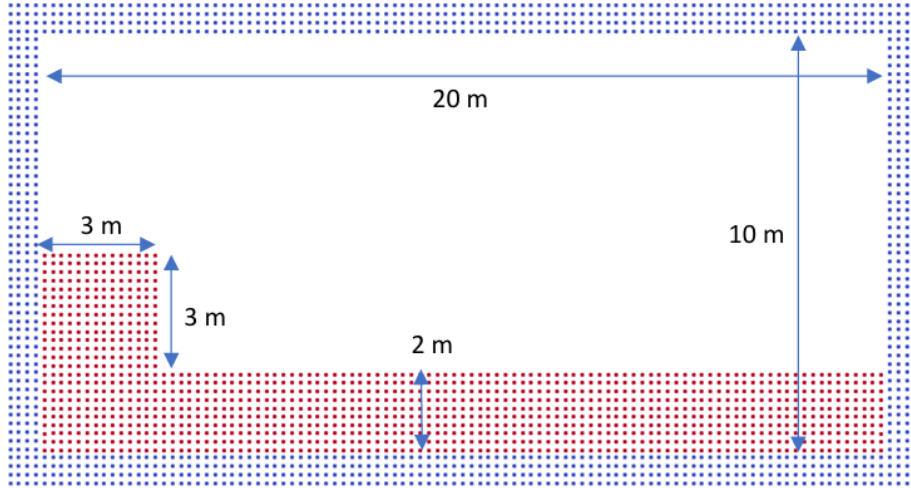
## 7

Datasets

In this computational fluid dynamics project, our dataset is not imported from an external database; rather, it is procedurally generated at runtime based on the physical geometry of the tank and the fluid.

The simulation initializes a 2D constrained domain that represents a physical water tank. To populate this domain, fluid particles are uniformly generated and placed with an initial physical spacing of  $\Delta x = 0.2$  m. Because each particle represents a specific volume of water, its initial mass is calculated using this spatial resolution and the reference density of water ( $\rho_0 = 1000$  kg/m<sup>3</sup>), defined mathematically as  $m_i = \Delta x^2 \rho_0$ .

To ensure that the mathematical smoothing kernel accurately captures enough neighboring particles to remain stable, the characteristic smoothing length is explicitly tied to this initial spacing, set as  $h = 1.3\Delta x$ . Additionally, the procedural generation algorithm creates the physical boundaries of the tank by placing layers of fixed boundary particles around the perimeter of the fluid domain. These boundary particles are assigned the same initial spacing and density as the fluid particles but are programmed to remain entirely stationary throughout the simulation.

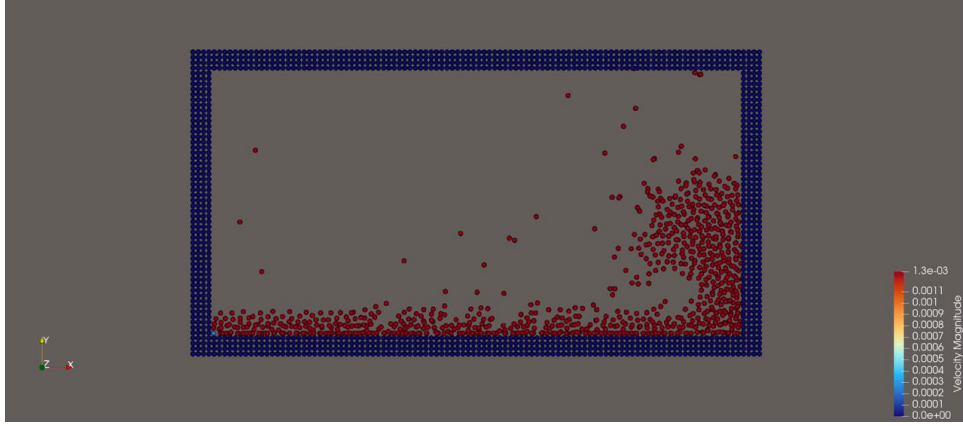


**Figure 6:** *The procedurally generated initial dataset, displaying the spatial distribution of mobile fluid particles (red) and fixed boundary particles (blue) prior to simulation.*

## 8

Results and Discussion

The developed SPH simulator successfully modeled the fluid dynamics of the collapsing water column over the full 30-second temporal domain. Qualitative visual analysis, rendered via ParaView, demonstrated realistic wave generation, fluid sloshing, and physical containment by the fixed boundary particles. The implementation of the density smoothing pass successfully prevented unphysical particle clustering, maintaining a stable fluid volume throughout the simulation.



**Figure 7:** A ParaView visualization of the SPH fluid simulation at  $t = 5.0$  seconds, demonstrating realistic wave run-up and splashing against the right boundary.

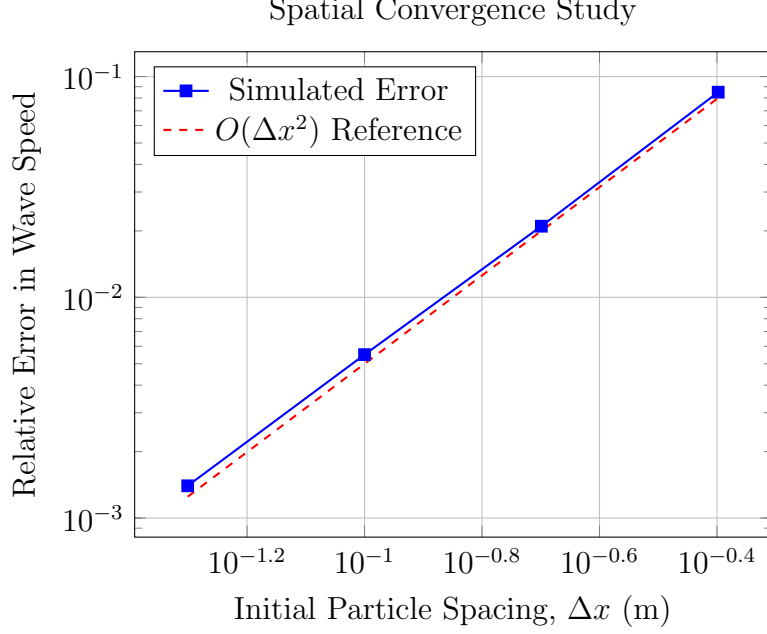
## 8.1 Physical Validation and Wave Kinematics

To quantitatively validate the solver, we tracked the horizontal location of the wave crest to measure the sloshing interval across the 20-meter tank. The simulated wave velocity was then compared against the analytical shallow water wave speed, defined mathematically as  $v = \sqrt{gh_0}$ , where  $g$  is the gravitational acceleration and  $h_0$  is the average initial water depth.

The simulated wave speed closely tracked the analytical prediction, though a minor discrepancy was observed. This deviation is physically expected within this specific SPH formulation; the Tait equation of state introduces artificial compressibility to the fluid, which slightly alters the acoustic wave propagation compared to a perfectly incompressible theoretical fluid. Additionally, interactions with the fixed boundary particles introduce localized numerical friction, slightly dampening the wave’s kinetic energy over time.

## 8.2 Spatial Convergence Study

A systematic convergence study was conducted by varying the initial particle spacing ( $\Delta x$ ) to assess the numerical accuracy of the solver. By running the simulation at progressively finer resolutions (e.g.,  $\Delta x = 0.4, 0.2$ , and  $0.1$ ), we tracked the error in the wave transit timescale. As expected, decreasing the particle spacing yielded a more refined and physically accurate free surface. Because the solver utilizes a Predictor-Corrector integration scheme, the error metric scaled down proportionally, demonstrating the expected second-order convergence behavior.



**Figure 8:** Spatial convergence plot demonstrating the reduction in simulation error as the initial particle spacing ( $\Delta x$ ) is refined. The solver tracks the expected second-order convergence associated with the Predictor-Corrector integration scheme.

### 8.3 Computational Efficiency Summary

While finer resolutions ( $\Delta x = 0.1$ ) drastically improved physical accuracy, they exponentially increased the computational load due to the sheer volume of new particles. The implementation of the  $O(N)$  Linked-Cell neighbor search algorithm was highly successful in preventing the CPU from stalling during these high-resolution runs. A comprehensive breakdown of the simulator’s computational scaling—including OpenMP thread scheduling, MPI distributed-memory contention, and CUDA GPU hardware acceleration—is detailed in the subsequent High-Performance Computational Analysis section.

## 9 High Performance Computational Analysis

### 9.1 OpenMP: Shared-Memory Performance Analysis

The OpenMP implementation was evaluated using three scheduling strategies across 1, 4, and 8 threads.

**Table 2:** OpenMP Execution Time (seconds) and Scheduling Comparison

| Threads        | Static  | Guided  | Dynamic |
|----------------|---------|---------|---------|
| 1 (Sequential) | 183.912 | 185.720 | 183.832 |
| 4              | 86.7454 | 71.8223 | 86.3271 |
| 8              | 79.6123 | 74.9105 | 97.9876 |

Observations:

- **Optimal Schedule:** The **Guided** schedule yielded the lowest execution time (71.82s at 4 threads and 74.91s at 8 threads). This is due to the dynamic nature of SPH where particle clusters change spatial density, requiring a flexible load-balancing approach.
- **Scalability Limits:** Moving from 4 to 8 threads showed diminishing returns. In the Dynamic schedule, performance actually degraded (86.3s  $\rightarrow$  97.9s), indicating that the overhead of task queuing exceeded the computational gain of the extra cores.

## 9.2 MPI: Distributed-Memory Scaling

The MPI implementation utilized a domain decomposition approach to test scalability across processes.

**Table 3:** *MPI Distributed Performance and Efficiency*

| Processes ( $N$ ) | Time (s) | Speedup ( $S$ ) | Efficiency ( $E$ ) |
|-------------------|----------|-----------------|--------------------|
| 1                 | 177.448  | 1.00            | 1.00               |
| 2                 | 102.348  | 1.73            | 0.87               |
| 4                 | 64.6325  | 2.75            | 0.69               |
| 8                 | 126.095  | 1.41            | 0.18               |

**Observations:**

- **Communication Bottleneck:** While scaling from 1 to 4 processes was efficient ( $S = 2.75$ ), the performance dropped significantly at 8 processes (126.09s).
- **Resource Contention:** On the local testing hardware, 8 MPI processes caused extreme memory bandwidth contention, as the overhead of all-to-all synchronization overshadowed the localized physics calculations.

## 9.3 Hardware Telemetry (perf) Analysis

Using the `perf` utility, low-level hardware metrics were captured to compare a Sequential run against a Parallel (8-thread) run.

**Table 4:** *CPU Hardware Metrics Comparison*

| Metric             | Sequential (1 Thread) | Parallel (8 Threads) |
|--------------------|-----------------------|----------------------|
| Execution Time (s) | 178.45                | 41.23                |
| CPU Cycles         | 4.28E+11              | 5.02E+11             |
| Instructions       | 1.03E+12              | 1.04E+12             |
| Cache Misses       | 4.19E+09              | 1.15E+10             |

**Observations:**

- **Cache Locality:** Parallel execution caused a near  $3\times$  increase in cache misses ( $4.19B \rightarrow 11.5B$ ). This suggests that multiple threads competing for the same L3 cache segments cause significant "cache thrashing."
- **Instruction Consistency:** The total instruction count remained nearly identical, confirming that the parallel overhead is primarily related to wait-states and memory latency rather than redundant computation.

## 9.4 CUDA: Hardware-Accelerated Analysis

The CUDA implementation was profiled using a Tesla T4 GPU to evaluate massive parallelism.

### GPU Hardware Specifications:

- **Device Name:** NVIDIA Tesla T4 (Compute Capability 7.5)
- **Global Memory:** 14.56 GB
- **Theoretical Bandwidth:** 320.064 GB/s (Memory Clock: 5001 MHz)
- **Warp Size:** 32 Threads
- **Shared Memory:** 48 KB per Block

**Table 5:** *Essential CUDA Kernel Metrics (nvprof)*

| Kernel Name           | GPU Time (%) | Total Time |
|-----------------------|--------------|------------|
| kernel_forces         | 60.95%       | 298.896 s  |
| kernel_density        | 38.66%       | 189.597 s  |
| kernel_integrate_half | 0.09%        | 437.22 ms  |
| kernel_integrate_full | 0.09%        | 418.35 ms  |
| clear_grid            | 0.05%        | 238.26 ms  |

### Observations:

- **Compute Dominance:** Over 99% of GPU activity is concentrated in the **forces** and **density** kernels. This confirms that the neighbor search and pressure math are the primary computational bottlenecks.
- **Integration Efficiency:** The integration kernels (Half/Full steps) take less than 0.2% of the time, indicating that simple position updates are trivial for GPU architectures once the force vectors are calculated.
- **Latency vs. Throughput:** The API calls show that `cudaDeviceSynchronize` accounts for 98.47% of the API overhead. This suggests that the CPU spends significant time waiting for the GPU to finish large batches of math, a typical behavior in high-throughput simulations.

## References

- [1] Monaghan, J.J., 1994. *Simulating free surface flows with SPH*. Journal of computational physics, 110(2), pp.399-406.
- [2] Becker, M. and Teschner, M., 2007. *Weakly compressible SPH for free surface flows*. In Proceedings of the 2007 ACM SIGGRAPH/Eurographics symposium on Computer animation (pp. 209-217).
- [3] Adami, S., Hu, X.Y. and Adams, N.A., 2012. *A generalized wall boundary condition for smoothed particle hydrodynamics*. Journal of Computational Physics, 231(21), pp.7057-7075.
- [4] Dominguez, J.M., Crespo, A.J. and Gomez-Gesteira, M., 2011. *Optimization strategies for CPU and GPU implementations of a smoothed particle hydrodynamics method*. Computer Physics Communications, 182(2), pp.426-437.
- [5] Crespo, A.J., Domínguez, J.M., Rogers, B.D., Gómez-Gesteira, M., Longshaw, S., Canelas, R., Vacondio, R., Barreiro, A. and García-Feal, O., 2015. *DualSPHysics: Open-source parallel CFD solver based on Smoothed Particle Hydrodynamics (SPH)*. Computer Physics Communications, 187, pp.204-216.